



Random Inputs for Breaking Things

Fuzz Testing



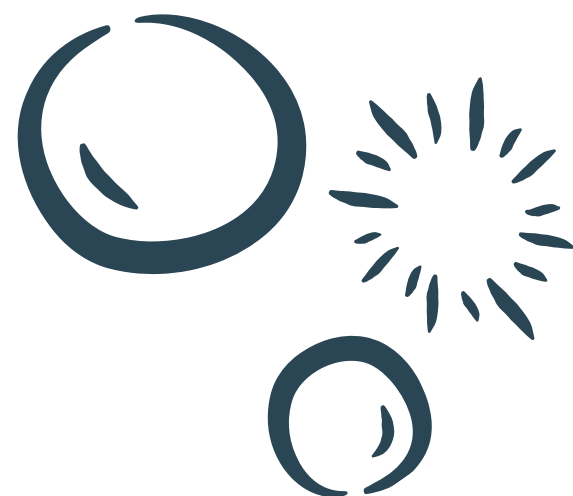
The feeling of random
The chaos of unpredictable





Outline

Covered topics



Introduction To Fuzz Testing

Why Use Fuzz Testing?

How Does Fuzz Testing Work?

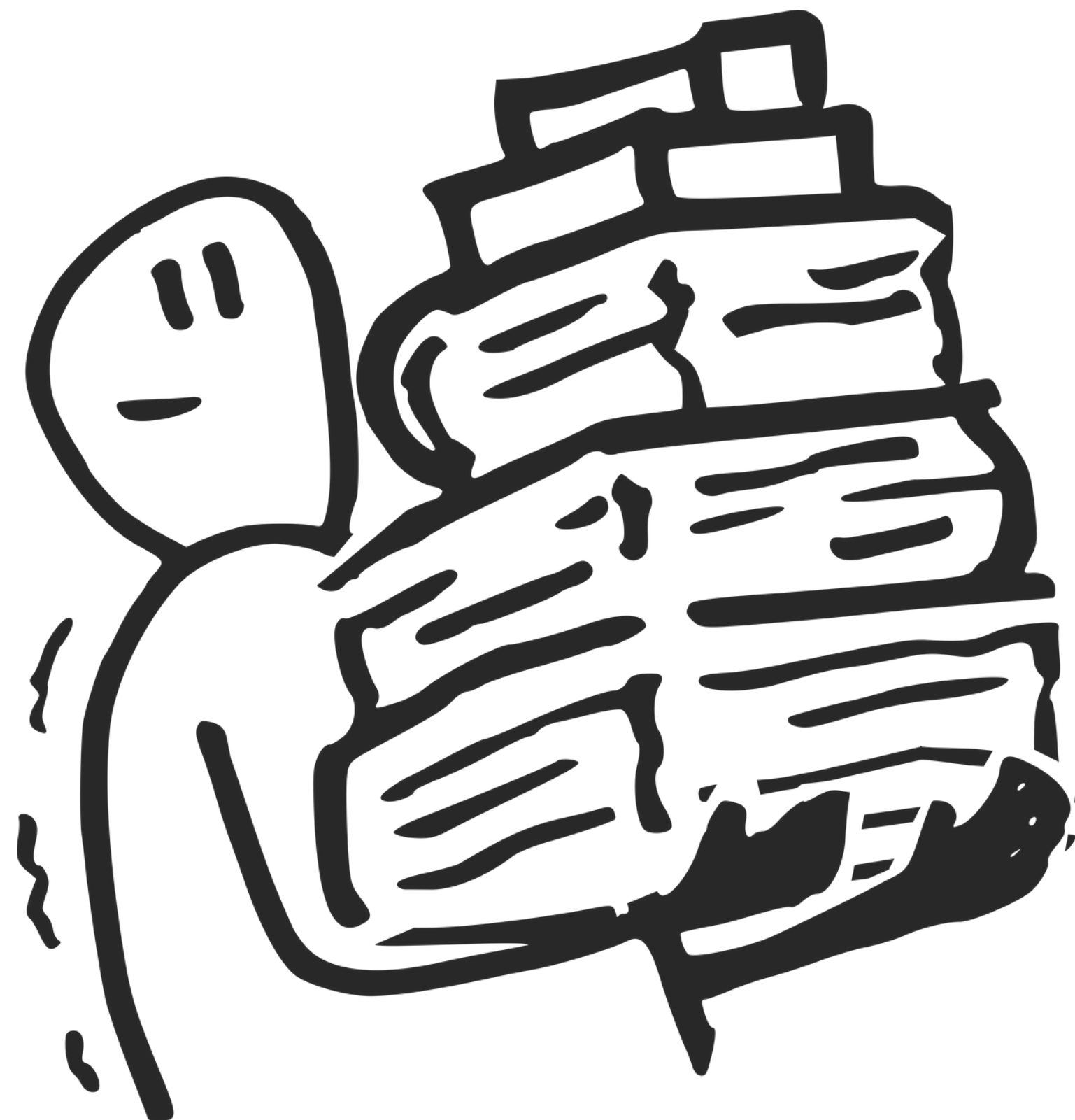
Types Of Fuzzing

Go Fuzzing



Definition of
Fuzz Testing

Introduction to Fuzz Testing





What is Fuzz Testing?

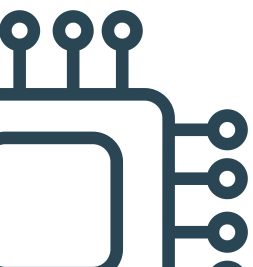
Fuzz Testing or Fuzzing is an automated **software testing technique** that injects invalid, malformed or **unexpected inputs** into a system, to reveal software **bugs or vulnerabilities**.



In a dark and stormy night

- 1988 class project
- Prof. **Barton Miller** sitting in his apartment
 - connected to his university computer via telephone line
- The thunderstorm caused noise on the line
- UNIX commands crashed

“I wanted a name that would evoke the feeling of random, unstructured data. After trying out several ideas, I settled on the term fuzz.”





Barton P. Miller, Lars Fredriksen and Bryan So

Study of the Reliability of UNIX Utilities

COMMUNICATIONS OF THE ACM/December 1990/Vol.33, No.12



TABLE II. List of Utilities Tested and the Systems on which They Were Tested (part 1)

●=utility crashed, ○=utility hung, *=crashed on SunOS 3.2 but not on SunOS 4.0, ⊕ = crashed only on SunOS 4.0, not 3.2. —=utility unavailable on that system. !=utility caused the operating system to crash.

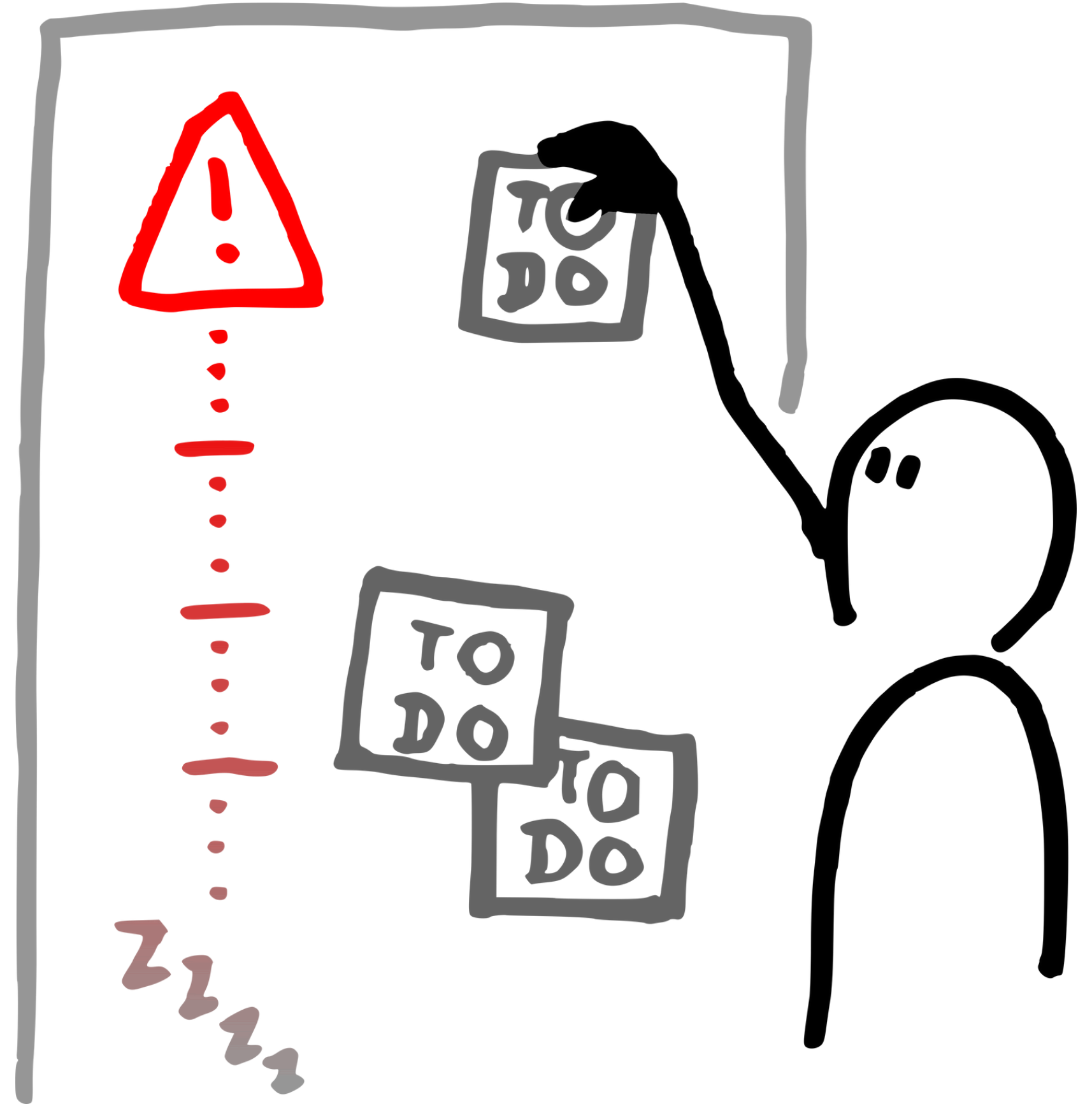
Utility	VAX (v)	Sun (s)	HP (h)	I386 (x)	AIX 1.1 (a)	Sequent (d)
adb	●○	●	●	○	—	—
as	●	—	—	●	●	●
awk	—	—	—	●	—	—
bc	—	—	—	●○	—	—
bib	—	—	—	—	—	—
calendar	—	—	—	—	—	—
cat	—	—	—	—	—	—
cb	●	—	—	—	—	—
cc	—	—	●	●	○	●
/lib/ccom	—	—	—	—	—	—
checkeq	—	—	—	—	—	●
checknr	—	—	—	—	—	—
col	●○	●	●	●○	●	●
colcrt	—	—	—	—	—	—
colrm	—	—	—	—	—	—
comm	—	—	—	—	—	—
compress	—	—	—	—	—	—
/lib/cpp	—	—	—	—	—	—
csh	●○	○	○	—	○	○
dbx	—	*	—	—	—	—
dc	—	—	—	○	—	—
deqn	—	—	—	—	—	—
deroff	●	●	—	—	—	—
diction	●	—	●	—	●	●
diff	—	—	●	—	—	●
ditroff	●○	●	—	—	—	—
dtbl	—	—	—	—	—	—
emacs	●	●	○	—	—	—
eqn	—	●	●	●	—	—
expand	—	—	—	—	—	—
f77	●	—	—	—	—	—
fmt	—	—	—	—	—	—

¹ Barton P. Miller, Lars Fredriksen, Bryan So. “Study of the reliability of UNIX utilities”. December 1990/Vol.33, No.12 Communications of the ACM.



Advantages and
Disadvantages

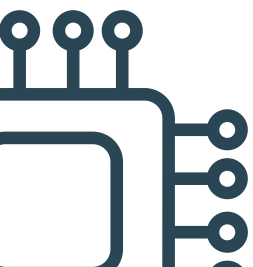
Why Use Fuzz Testing?





Advantages

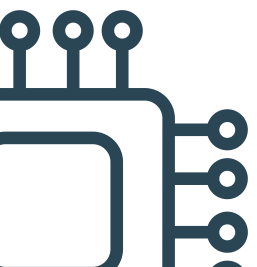
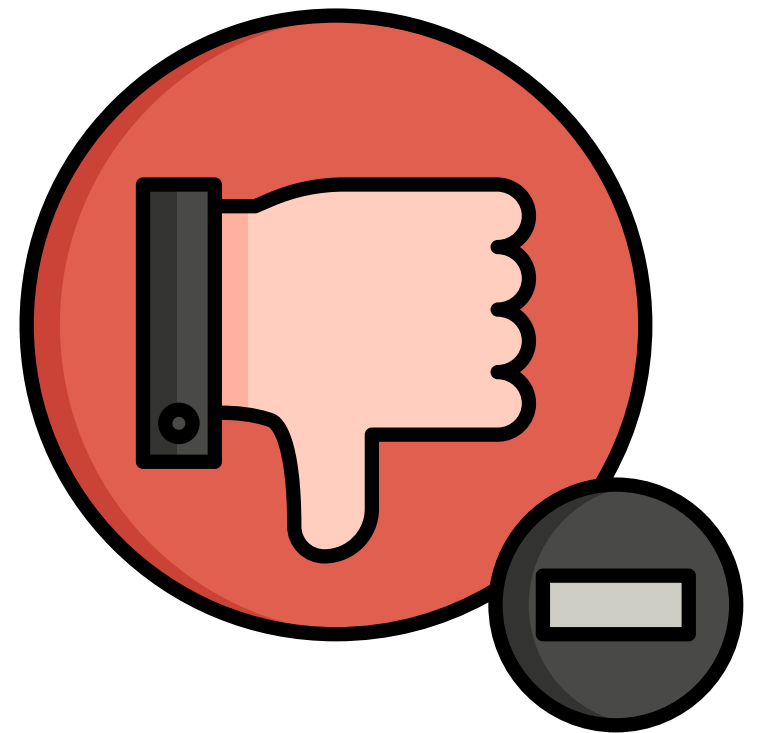
- Uncover **hidden bugs**
 - detect bugs that traditional testing methods might miss
 - memory corruption issues, input validation problems and crashes
- Automation
- **Scalability**
 - a wide range of inputs in a short period
- **Simple implementation**
- Numerous **open-source** tools
 - AFL (American Fuzzy Lop), LibFuzzer, Honggfuzz, ...





Disadvantages

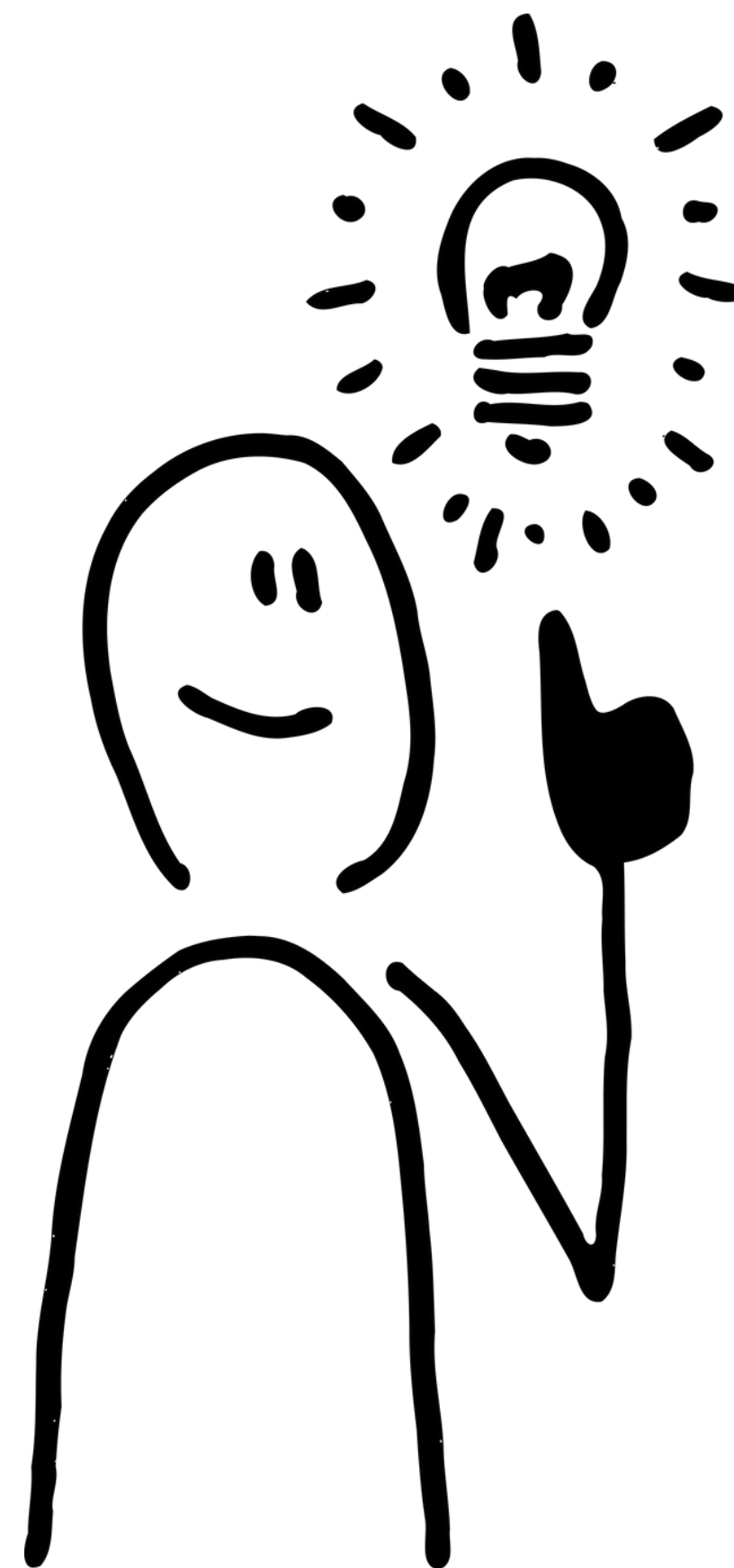
- Fuzzing is **not** the *silver bullet* of software security
 - Unit testing and integration testing are necessary
- Find defects that result in crashes
 - SQL Injection, XSS, ...
- **False positive** and false negative
- **Memory and CPU intensive**





Fuzz Testing
Vital Steps

How Does Fuzz Testing Work?





Identification Of
Target System





Identification Of
Target System

Determination
Of Inputs





Identification Of
Target System

Determination
Of Inputs

Generation Of
Fuzzed Data





Identification Of
Target System

Determination
Of Inputs

Generation Of
Fuzzed Data

Execution Of Test
With Fuzzed Data





Identification Of
Target System

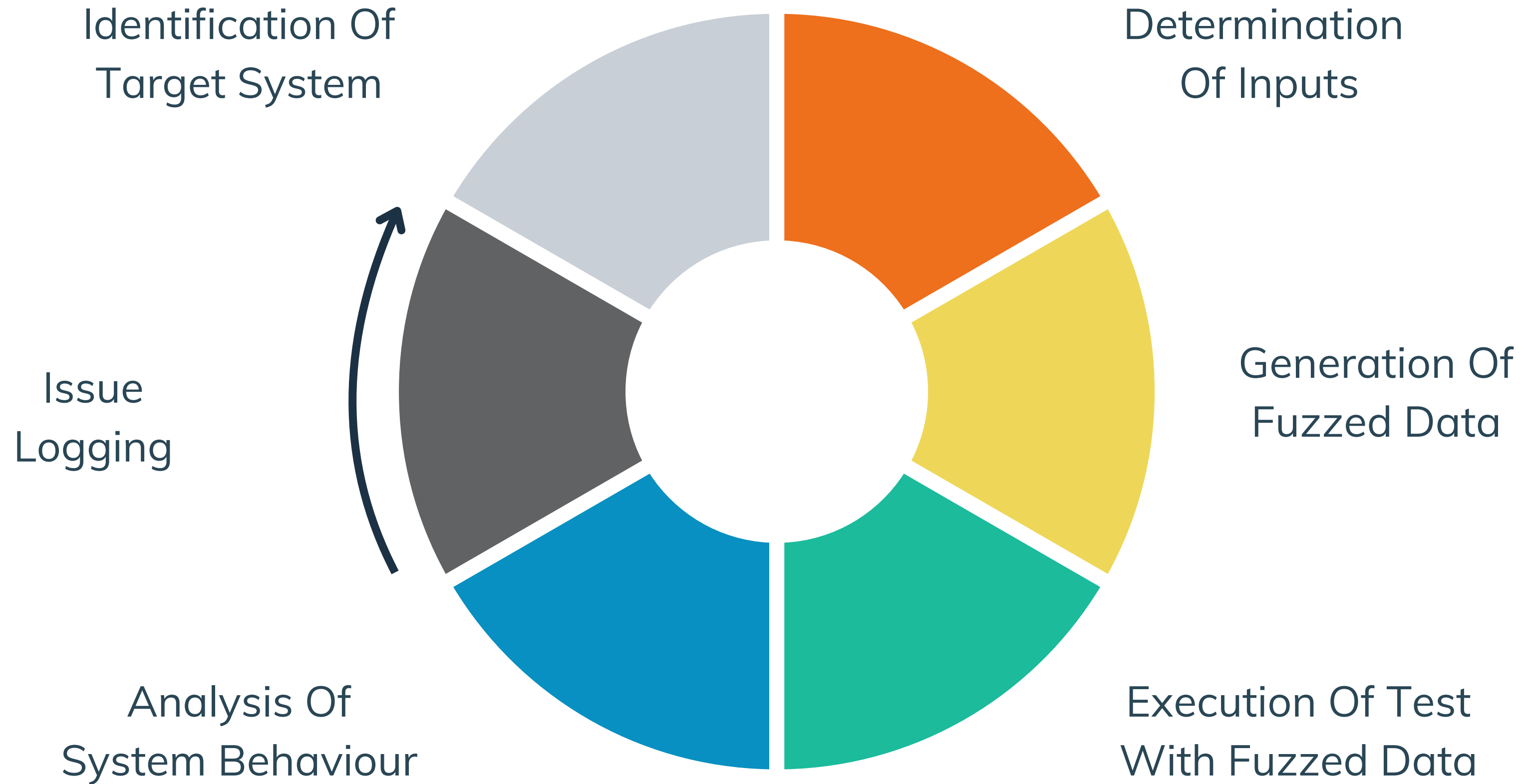
Determination
Of Inputs

Generation Of
Fuzzed Data

Execution Of Test
With Fuzzed Data

Analysis Of
System Behaviour







From dumb
to smart

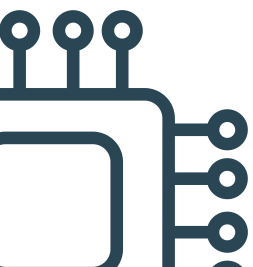
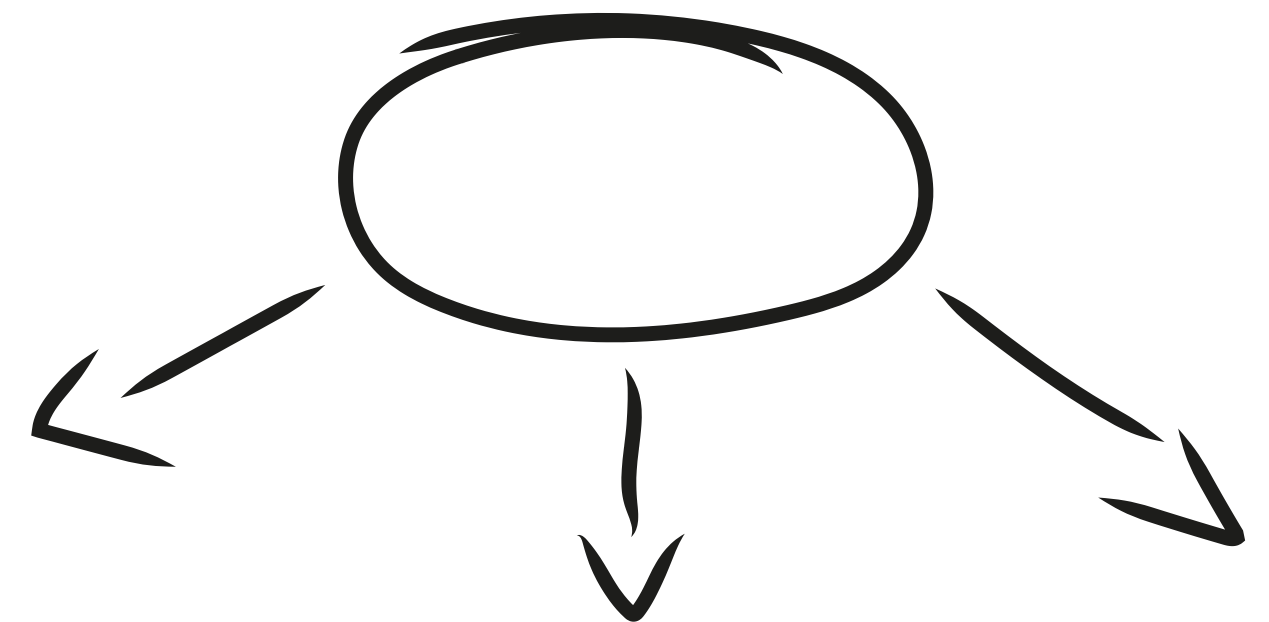
Types Of Fuzzing





Types

- A **fuzzer** can be categorized in several different ways:
 - how the inputs are generated
 - generation-based or mutation-based
 - how much they know the structure of the program
 - black-box, white-box, grey-box
 - how much they know the structure of the inputs
 - protocol, file format, grammar-based



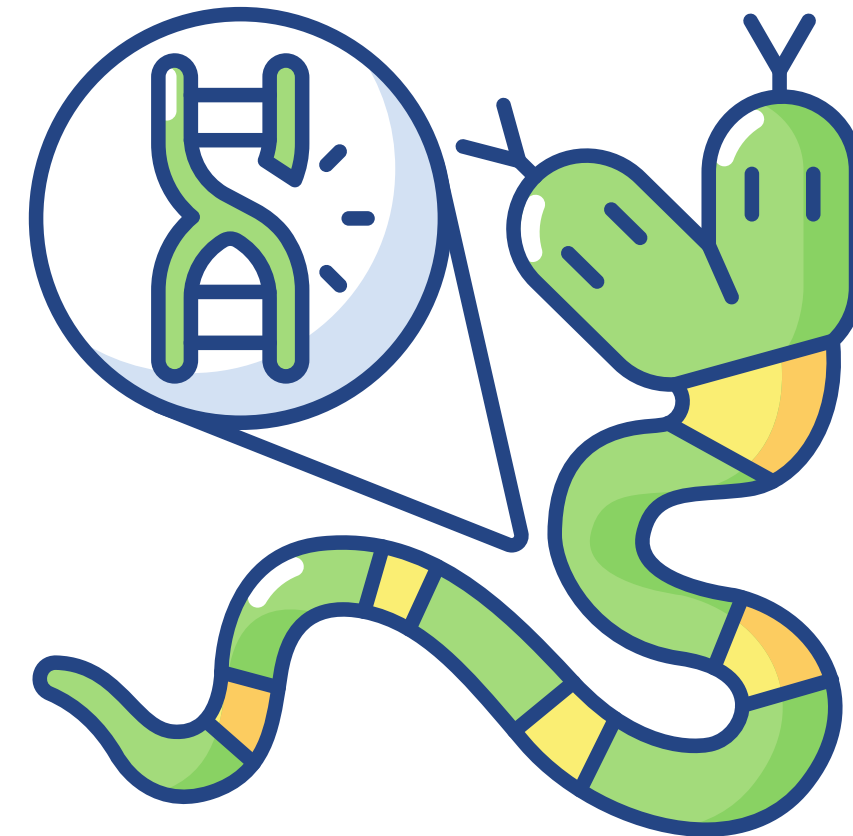


Input generation

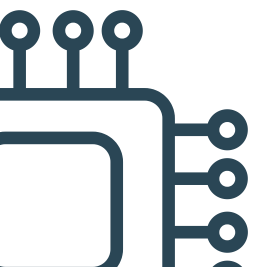
- From scratch or by modifying existing ones



Generation-based



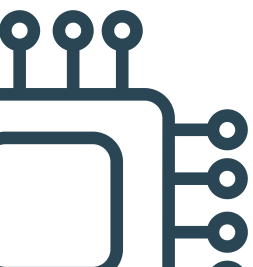
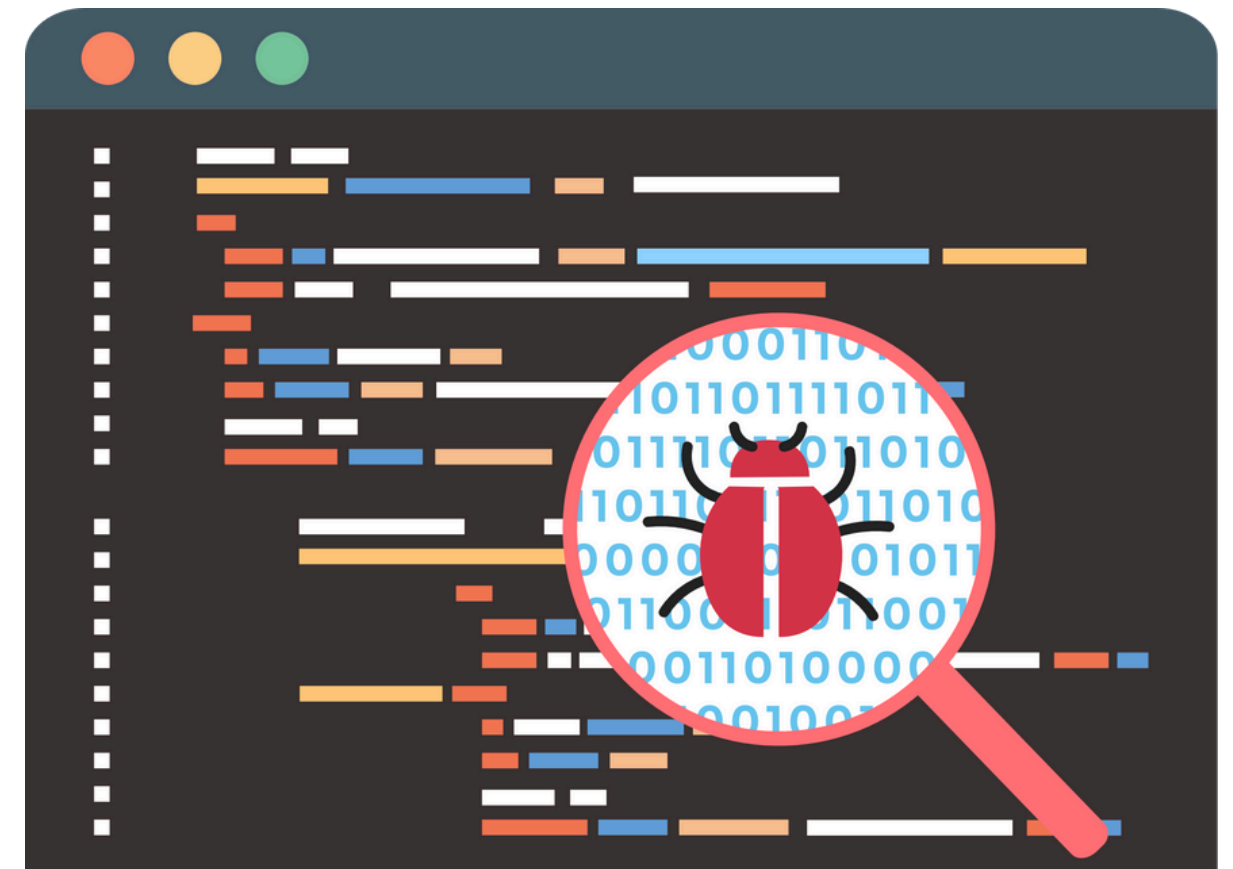
Mutation-based





Knowledge of program structure

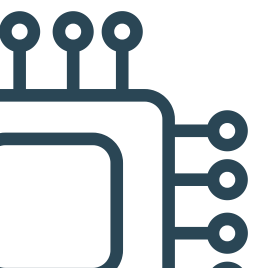
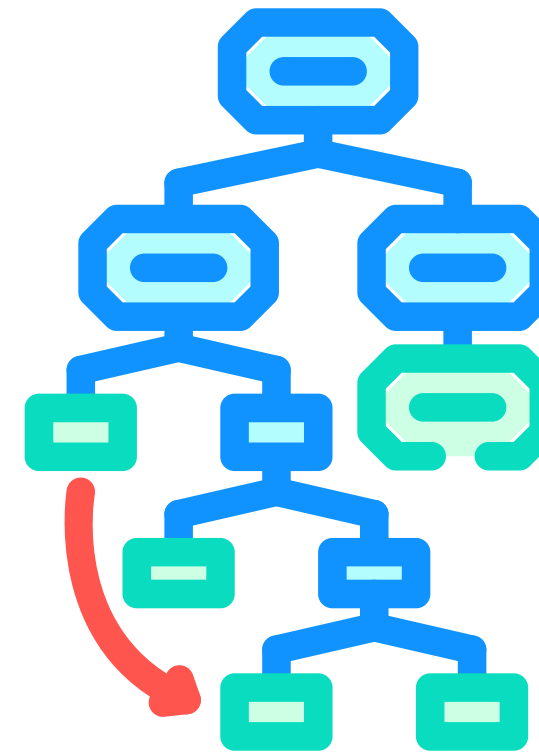
- More effective fuzzer if it achieves high degree of **code coverage**
 - **black-box**
 - unaware of internal program structure
 - fast. but only shallow bugs
 - **white-box**
 - full knowledge of the system
 - it can become prohibitive
 - **grey-box**
 - reasonable compromise





Knowledge of input structure

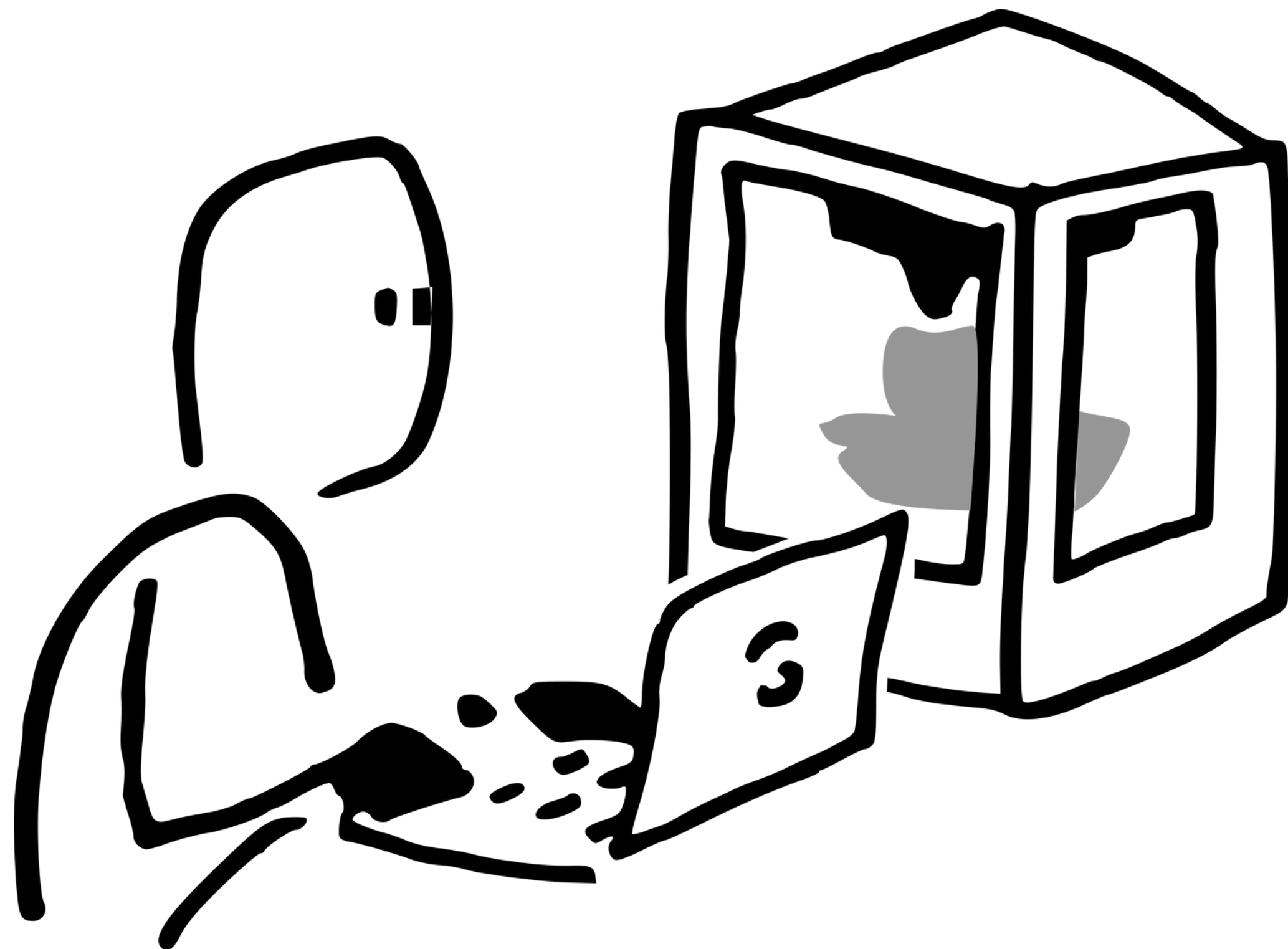
- Leverage the input model to generate inputs
 - **smart**
 - greater valid inputs (e.g. syntax tree, formal grammar)
 - **dumb**
 - lower proportion of valid inputs
 - flip random bits
 - delete blocks of data
 - ...





Validating Credentials with Fuzz Testing

Go Fuzzing





GoFuzzBuster

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

GoFuzzBuster Public

[Pin](#) [Unwatch](#) 1 [Fork](#) 0 [Star](#) 0

main 1 Branch 0 Tags

Go to file t Add file Code

AgneseRe

doc: first version readme

787bf7f · 3 days ago

5 Commits

testdata/fuzz/FuzzAuthHandler

code: add constraints username and password

last week

README.md

doc: first version readme

3 days ago

auth_handler.go

code: add constraints username and password

last week

auth_handler_test.go

code: add constraints username and password

last week

README

GoFuzzBuster

This project implements *fuzz testing* for an authentication handler function, `AuthHandler`. The main goal of the fuzz

About

No description, website, or topics provided.

[Readme](#) [Activity](#) [0 stars](#) [1 watching](#) [0 forks](#)

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

¹ <https://github.com/AgneseRe/GoFuzzBuster>



Thank You For Your Attention!

